TEMP

A. THE BACKLOG TASK

Epic / Feature Name: Partner Webhook Ingestion Pipeline
Ticket ID & Title: `[BACKEND-501] Implement Custom Route Constraint for Partner IDs`

User Story:
"As an external B2B partner, I want to send API requests to a strongly validated URL so that only properly formatted partner IDs are processed by the ingestion engine."

Business Context & Technical Requirements:
We are building a lightweight webhook receiver. Partners will hit our endpoint with their unique `partnerId` in the URL.
A valid Partner ID at our company **must** follow this strict business rule:

1. It must be exactly 8 characters long.
2. It must start with the prefix `"PTN"` (case-insensitive).
   (Example valid IDs: `ptn12345`, `PTN99999`)

Instead of cluttering our route templates with massive inline regex strings, or polluting our endpoint logic with structural string checks, I want you to encapsulate this rule into a **Custom Route Constraint**.

Scaffolding & Project Setup:

- Use the **ASP.NET Core Empty** project template.
- Create a folder named `CustomConstraints`.
- Place your constraint class inside this folder (e.g., `PartnerIdConstraint.cs`).
- Do all your routing and pipeline registration inside `Program.cs`.

Acceptance Criteria (AC):

- [x] ~~Create a `PartnerIdConstraint` class that implements `IRouteConstraint`.~~
- [x] ~~Implement the `Match()` method to enforce the 8-character length and `"PTN"` prefix rules.~~
- [x] ~~Register the constraint in `Program.cs` under the alias `"partner"`.~~
- [x] ~~Create an endpoint using `endpoints.MapPost` (or `MapGet` if you prefer testing via browser) at the route: `/api/webhooks/{partnerId:partner}`.~~
- [x] ~~If the route matches (valid ID), write back a success message: `"Webhook received for partner: {partnerId}"`.~~
- [x] ~~Implement a global fallback middleware (`app.Run`) at the end of the pipeline. If a partner sends an invalid ID (e.g., `/api/webhooks/ABC12345`), the route should fail the constraint,~~

~~skip the endpoint, and the fallback should return a `404 Not Found` with the text `"Invalid URL or Partner ID format."`~~

Definition of Done (DoD):

- Proper null-checking when pulling values from the `RouteValueDictionary`.
- No runtime exceptions if the URL parameter is completely missing.
- Clean C# conventions.

B. THE STANDUP INTERVIEW QUESTION

Type 3 (System Design & Trade-offs)

Context:
Hey, I was looking over the architecture for an upcoming feature. We need to accept a `countryCode` in our URL (e.g., `/api/shipping/{countryCode}`). This code must match one of the 195 valid ISO country codes (like `US`, `GB`, `FR`).

Question:
If we want to reject invalid country codes, should we build a **Custom Route Constraint** (like you are doing in ticket BACKEND-501) to validate the 195 codes, OR should we just accept any string in the route (`{countryCode:alpha}`) and validate the specific 195 codes **inside the endpoint's C# logic**?
Explain the trade-offs of both approaches, specifically regarding API User Experience (HTTP Status Codes) and Separation of Concerns.

Your turn!
Whenever you're ready, reply with your code for `BACKEND-501` (just paste the C# files) and your answer to the standup question. I'll review it like a real PR!